

Machine-learning model

The machine-learning component is a proof of concept for automatically analysing dashboard images for compliance with the IBCS rule concerning the unification of scenarios. The purpose is to reduce the amount of manual work required from IBCS experts.

Instead of checking the complete IBCS framework, the model focuses on ten possible scenario-related mistakes:

- Incorrect Actual graph notation
- Incorrect Previous Year graph notation
- Incorrect Plan graph notation
- Incorrect Forecast graph notation
- Missing or incorrect AC abbreviation
- Missing or incorrect PY abbreviation
- Missing or incorrect BU abbreviation
- Missing or incorrect PL abbreviation
- Missing or incorrect FC abbreviation
- Incorrect semantic scenario notation on chart elements or axes

The model uses anomaly labels. A label value of 1 means that a mistake was detected for that rule. A value of 0 means that no mistake was detected, the rule is compliant, or the rule is not applicable or not present in the dashboard.

Chosen approach

A multi-label classification approach was selected because one dashboard can contain multiple IBCS mistakes at the same time. A normal multi-class classifier would only select one category, while this model must evaluate all ten rules independently.

The current model is therefore a supervised multi-label image classifier that is used as an anomaly detector. It does not learn completely unknown anomalies. It learns the ten predefined anomaly labels included in the dataset.

A pretrained ResNet18 convolutional neural network is used through transfer learning. ResNet18 was chosen because it already contains image-recognition knowledge learned from a large dataset and provides a practical balance between model performance, training time and model size. This is especially useful because the available IBCS dataset is small.

The original ResNet18 output layer was replaced with:

- A dropout layer with a dropout rate of 0.3
- A linear layer with ten outputs

Each output represents one of the ten IBCS anomaly labels. All layers of ResNet18 are included in the Adam optimiser, meaning the complete network is fine-tuned rather than only training the final layer.

Dataset and labels

The notebook reads the label information from:

`../images_data/trial-correct.csv`

The dashboard images are loaded from:

`../images_data/ibcs-images`

The CSV contains an image path and the following ten label columns:

AC-graph

PY-graph

PL-graph

FC-graph

AC-abr

PY-abr

BU-abr

PL-abr

FC-abr

Axis

The compliant images generally contain zero values because no anomaly has been labelled. The non-compliant dashboards can contain one or multiple positive anomaly labels.

The dataset is divided using the following proportions:

- 70% training data
- 15% validation data
- 15% test data

The split is created with PyTorch's `random_split`. No random seed is currently set, so running the notebook again may produce a different train, validation and test split and therefore different results.

Image preprocessing

Every image is:

1. Loaded using Pillow.
2. Converted to RGB.
3. Resized to 224×224 pixels.
4. Converted into a PyTorch tensor.

The implemented transformation is:

```
transform = transforms.Compose([  
    transforms.Resize((224, 224)),
```

```
transforms.ToTensor()  
])
```

Although the notebook text mentions data augmentation, the final code does not apply rotations, flips, brightness adjustments or other augmentation. It also does not apply ImageNet normalisation, even though ImageNet-pretrained ResNet18 weights are used.

The same transformation must be used when making predictions unless the model is retrained with a different preprocessing pipeline.

Training configuration

The model uses the following settings:

Image size: 224 × 224

Batch size: 16

Maximum epochs: 30

Learning rate: 0.0001

Early stopping: 5 epochs

Threshold: 0.5

CUDA is used when a compatible GPU is available. Otherwise, the model runs on the CPU.

BCEWithLogitsLoss is used as the loss function. This loss function is suitable for multi-label classification because it evaluates every anomaly label independently.

The Adam optimiser updates all model parameters:

```
optimizer = optim.Adam(  
    model.parameters(),  
    lr=0.0001  
)
```

During training, the model produces ten raw output values called logits. BCEWithLogitsLoss internally combines the sigmoid operation with binary cross-entropy loss.

During prediction, sigmoid is applied manually. Every probability above 0.5 is classified as an anomaly:

```
predictions = torch.sigmoid(outputs) > 0.5
```

Training process

For every epoch, the model:

1. Loads dashboard images in batches.
2. Produces ten predictions for each image.
3. Compares the predictions with the ten labels.

4. Calculates the loss.
5. Performs backpropagation.
6. Updates the model weights using Adam.
7. Evaluates the model on the validation set.

Training and validation loss and label-level accuracy are recorded after every epoch.

Early stopping is based on validation loss. Whenever the validation loss improves, the model weights are copied and saved. Training should stop when validation loss does not improve for five consecutive epochs. In the recorded run, validation loss continued improving frequently enough that all 30 epochs were completed.

After training, the best-performing weights are restored before test evaluation.

Evaluation results

The recorded training run reached:

Training accuracy: approximately 99–100%

Validation accuracy: approximately 90%

Test accuracy: 90%

However, the reported accuracy represents the percentage of individual label values predicted correctly. Because most labels contain zeros, this result can appear high even when the model does not detect several rare anomalies.

The classification report shows that the model performed well mainly for:

- AC-graph: F1-score of 1.00 on seven positive test examples
- AC-abr: F1-score of 0.94 on nine positive test examples

For the remaining labels, the model produced an F1-score of zero in this test split. Several rules had only one to three positive test examples, while FC-graph and Axis had no positive test examples at all.

The overall results were:

Micro F1-score: 0.71

Macro F1-score: 0.19

Weighted F1-score: 0.57

Sample F1-score: 0.52

This means the 90% test accuracy should not be interpreted as the model successfully detecting all ten rules. The current model mainly demonstrates that the approach can learn some frequently represented anomalies, especially Actual graph and abbreviation mistakes.

The main limitation is the small and imbalanced dataset. Some mistakes, particularly Plan and Forecast scenarios, are uncommon or absent in the test data. The model therefore has insufficient examples to learn and evaluate these labels reliably.

Feedback generation

The model itself does not generate written feedback. It only returns ten probabilities.

A Python dictionary connects every model output to a fixed business explanation. For example, when AC-graph is detected, the application returns:

The dashboard does not correctly represent Actual values using dark solid notation.

The prediction function displays:

- The detected rule
- The anomaly status
- The model probability as a confidence percentage
- The predefined feedback message

When none of the ten outputs reaches the threshold of 0.5, the function reports that no clear anomaly was detected and that the dashboard is likely compliant.

This does not guarantee that the dashboard is compliant. It only means that the model did not detect one of the learned anomalies above the selected threshold.

Saved model

The final model is saved as:

`ibcs_final_model.pth`

Only the PyTorch state_dict is stored. This contains the learned model weights, but not the complete model architecture, label definitions, preprocessing steps or feedback messages.

To load the model correctly, a future developer must:

1. Recreate a ResNet18 model.
2. Replace its final layer with dropout 0.3 and a linear layer with ten outputs.
3. Load the saved state dictionary.
4. Apply the same image transformation.
5. Maintain the exact label order.
6. Apply sigmoid and the 0.5 threshold.
7. Connect predictions to the feedback dictionary.

The checkpoint should ideally be improved in the future by also storing the labels, thresholds, image size and model configuration.

Technology stack

The model was created using:

- Python
- Jupyter Notebook
- PyTorch

- Torchvision
- ResNet18
- Pillow
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Git and GitHub
- CUDA when available

Important limitations and future work

The current implementation is a proof of concept and not a production-ready compliance checker.

The most important limitations are:

- The dataset is strongly imbalanced between the ten anomaly labels.
- Some labels are absent from the test set.
- The train/test split is not reproducible because no random seed is set.
- The split is not stratified.
- No actual data augmentation is implemented.
- ImageNet normalisation is not applied.
- One general threshold of 0.5 is used for every rule.
- The model only predicts predefined anomalies.
- The model does not identify where in the dashboard the mistake occurs.
- A missing Plan or Forecast scenario is difficult to detect from an image alone.
- The reported overall accuracy is misleading without the per-rule metrics.

Further development should focus primarily on collecting and correctly labelling more dashboard images for every rule. The dataset should contain enough positive and negative examples for Actual, Previous Year, Plan, Forecast, abbreviations and semantic notation.

A reproducible and preferably stratified multi-label split should then be created. Data augmentation, ImageNet normalisation, class weighting and separate thresholds per rule should also be tested.

The model should be evaluated using precision, recall and F1-score per rule rather than relying on overall accuracy. A separate external test set containing dashboard styles and sources that were never used during training would provide a more reliable indication of real-world performance.